Reporte Técnico de Actividades Práctico-Experimentales Nro. 002

Comparación de modelos determinísticos, estocásticos, discretos y continuos

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Emilio Josue Flores Gallardo
Asignatura	Simulación
Ciclo	5
Unidad	U1: Introducción a la Simulación
Resultado de aprendizaje de la unidad	R1. Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	002
Título de la Práctica	Comparación de modelos determinísticos, estocásticos, discretos y continuos
Nombre del Docente	José O. Guamán Q.
Fecha	08 y 09 de octubre de 2026
Horario	Jueves 10h30 a 11h30 / Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas
Repositorio Git	Repositorio GitHub

2. Objetivo(s) de la Práctica

Objetivo general: diferenciar los principales paradigmas de simulación mediante la construcción de modelos computacionales.

Objetivos específicos:

- Implementar un modelo determinístico y uno estocástico.
- Comparar modelos discretos y continuos.
- Analizar el efecto de la aleatoriedad sobre los resultados.

3. Materiales, Reactivos, Equipos y Herramientas

Software y librerías:

- Python 3.12 con entorno virtual (`venv`) y librerías listadas en `requirements.txt`.
- Librería **NumPy** para el manejo de arreglos, cálculo exponencial y generación de ruido aleatorio con distribución normal.

- Librería **Matplotlib** para generar las gráficas comparativas de los cuatro modelos.
- Librerías complementarias de la guía (`scipy`, `pandas`, `pynetlogo`) instaladas en el entorno virtual.
- Visual Studio Code como editor de código, junto con Git y GitHub para el control de versiones.

Equipos: computadora con mínimo 4 GB de RAM, procesador de doble núcleo o superior, sistema operativo Linux, macOS o Windows y conexión a Internet para instalar las herramientas.

4. Procedimiento / Metodología Ejecutada

4.1. Fundamento teórico

En esta práctica se estudia el crecimiento poblacional comparando cuatro modelos de simulación distintos a partir del arreglo de datos de la guía:

$$\texttt{datos} = [1000,\ 1100,\ 1250,\ 1400,\ 1600,\ 1850,\ 2100] \tag{1}$$

Cada modelo representa el cambio de la población P en el tiempo t bajo un enfoque diferente (Tabla 4.1):

1. **Modelo determinístico:** calcula la población de forma exacta sin usar azar mediante la ecuación de crecimiento exponencial:

   $$P(t) = P_0 e^{rt} \tag{2}$$

   donde P_0 es la población inicial, r la tasa de crecimiento y t el tiempo.

2. **Modelo discreto:** actualiza la población en períodos enteros definidos ($t = 1, 2, \ldots,$ tiempo), sumando en cada paso el incremento proporcional a la población actual:

   $$P_{t+1} = P_t + rP_t \tag{3}$$

3. **Modelo estocástico:** le suma al cálculo discreto una perturbación aleatoria ϵ_t con distribución normal de media 0 y desviación estándar σ:

   $$P_{t+1} = P_t + rP_t + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \sigma^2) \tag{4}$$

   Si por efecto del ruido el valor cae por debajo de cero ($P_{t+1} < 0$), el algoritmo fija $P_{t+1} = 0$ para no tener poblaciones negativas.

4. **Modelo continuo (método de Euler):** aproxima la ecuación diferencial $\frac{dP}{dt} = rP$ dividiendo el tiempo en pasos pequeños Δt (dt):

   $$P_{t+\Delta t} = P_t + \Delta t(rP_t) \tag{5}$$

Tabla 1. Resumen de los cuatro modelos de simulación implementados.

Modelo	Ecuación principal	Característica en la simulación
Determinístico	$P(t) = P_0 e^{rt}$	Sin aleatoriedad; repite siempre la misma curva exacta para un mismo valor de r.
Discreto	$P_{t+1} = P_t + rP_t$	Avanza en saltos enteros de tiempo (período por período).
Estocástico	$P_{t+1} = P_t + rP_t + \epsilon_t$	Incluye ruido normal $\mathcal{N}(0, \sigma^2)$ y corrige valores menores a cero.
Continuo (Euler)	$P_{t+\Delta t} = P_t + \Delta t(rP_t)$	Usa un paso pequeño dt para aproximar la derivada continua $\frac{dP}{dt} = rP$.

4.2. Las 7 etapas de un proyecto de simulación

Para el desarrollo de la práctica se siguieron las siete etapas de un proyecto de simulación, aplicadas a la implementación y comparación de los cuatro modelos de crecimiento poblacional en Python:

- **Etapa 1. Definición del problema**
  Se planteó la necesidad de simular la dinámica de crecimiento y decrecimiento de una población a partir de un conjunto de observaciones históricas, con el objetivo de comparar el comportamiento del sistema bajo cuatro paradigmas: determinístico, discreto, estocástico y continuo.

- **Etapa 2. Formulación conceptual**
  Se definieron las ecuaciones matemáticas de cada enfoque en función de la población inicial (P_0), la tasa de crecimiento (r) y el tiempo (t): crecimiento exponencial exacto $P(t) = P_0 e^{rt}$ para el modelo determinístico; recurrencia por períodos $P_{t+1} = P_t + rP_t$ para el modelo discreto; adición de perturbación aleatoria normal $P_{t+1} = P_t + rP_t + \epsilon_t$ con $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$ para el modelo estocástico; y aproximación numérica de Euler $P_{t+\Delta t} = P_t + \Delta t(rP_t)$ sobre la ecuación diferencial $\frac{dP}{dt} = rP$ para el modelo continuo.

- **Etapa 3. Recolección y análisis de datos**
  Se utilizó como referencia el arreglo de entrada establecido en la guía de la práctica, `datos = [1000, 1100, 1250, 1400, 1600, 1850, 2100]`, tomando como condición inicial $P_0 = 1000$ individuos a lo largo de un horizonte de 6 períodos de simulación.

- **Etapa 4. Construcción del modelo**
  Se implementó el sistema en Python combinando la arquitectura Modelo–Vista–Controlador (MVC) con el patrón de diseño *Factory Method* (`factory/`, `controller/` y `view/`), encapsulando cada paradigma matemático en su propio módulo y utilizando `NumPy` para el cálculo vectorial y la generación de variables aleatorias normales.

- **Etapa 5. Verificación y validación**
  Se comprobó la ejecución de los cuatro modelos bajo las mismas condiciones iniciales, verificando el cumplimiento de la restricción biológica de no negatividad en el modelo estocástico (si $P_{t+1} < 0 \Rightarrow P_{t+1} = 0$), la convergencia del método de Euler al reducir el paso Δt y la correspondencia de las trayectorias simuladas frente al arreglo de datos base.

- **Etapa 6. Experimentación y análisis de resultados**
  Se ejecutaron escenarios comparativos modificando el signo y magnitud de la tasa intrínseca r (crecimiento con $r > 0$ y decrecimiento con $r < 0$), el nivel de dispersión

estocástica σ y el tamaño de paso Δt, generando las gráficas superpuestas mediante `Matplotlib` para contrastar las cuatro curvas.

- **Etapa 7. Implementación y documentación**
  Se organizó el código fuente en el repositorio de GitHub junto con los archivos de soporte del entorno virtual (`requirements.txt`, `.gitignore` y `README.md`), documentando la interpretación matemática de los resultados y las conclusiones de la práctica.

4.3. Arquitectura MVC y patrón de diseño Factory Method

A diferencia de un script lineal donde todas las funciones quedan sueltas en un mismo archivo, para esta segunda práctica decidí estructurar el código combinando la arquitectura **Modelo–Vista–Controlador (MVC)** con el patrón de diseño creacional **Factory Method** dentro del paquete `factory/`. Esta decisión permite desacoplar por completo la lógica matemática de cada paradigma respecto al controlador y a la interfaz gráfica, siguiendo buenas prácticas de *Clean Code* y los principios **SOLID** (Tabla 3):

- **Principio de Responsabilidad Única (SRP):** Cada modelo matemático vive en su propio archivo independiente (`deterministic.py`, `discreet.py`, `stochastic.py` y `continuous.py`). Así, si hace falta corregir la fórmula de Euler o ajustar la restricción de no negatividad del modelo estocástico, solo se toca esa clase sin riesgo de romper los demás algoritmos.
- **Principio Abierto/Cerrado (OCP) mediante Factory Method:** La clase fábrica centralizada en `factory/factory.py` se encarga de instanciar el modelo solicitado según el tipo requerido por el controlador. Si más adelante se pide agregar un quinto modelo (por ejemplo, crecimiento logístico o Runge-Kutta), basta con crear la nueva clase y registrarla en la fábrica sin modificar el flujo de `controller.py`.
- **Separación entre cálculo y visualización:** El controlador (`controller/controller.py`) coordina la ejecución de los cuatro modelos usando los mismos parámetros de entrada y le entrega las listas de tiempo y población ya procesadas a la vista (`view/graphics.py`), la cual se dedica únicamente a armar las gráficas comparativas en Matplotlib.

Tabla 2. Estructura modular del proyecto APE 002 (MVC + Factory Method).

4.4. Parámetros y pasos de ejecución

En la Tabla 4.4 se detallan los parámetros configurados para comparar los cuatro modelos tomando como base el arreglo `datos = [1000, 1100, 1250, 1400, 1600, 1850, 2100]`, donde la población pasa de 1000 a 2100 en 6 períodos (lo que equivale a una tasa promedio cercana a $r = 0{,}13$).

Tabla 3: Estructura modular del proyecto APE 002 (MVC + Factory Method).

Archivo / Módulo	**Responsabilidad dentro de la arquitectura**
`factory/deterministic.py`	Implementa el cálculo exponencial exacto $P(t) = P_0 e^{rt}$ sobre el vector de tiempo.
`factory/discreet.py`	Calcula la evolución por períodos enteros mediante $P_{t+1} = P_t + rP_t$.
`factory/stochastic.py`	Suma el ruido normal $\mathcal{N}(0, \sigma^2)$ en cada período y controla que la población no sea menor a 0.
`factory/continuous.py`	Resuelve la ecuación diferencial por el método numérico de Euler usando el paso dt.
`factory/factory.py`	Implementa el patrón *Factory Method* para crear y devolver la instancia de cada modelo.
`controller/controller.py`	Pide a la fábrica los cuatro modelos, corre las simulaciones con los mismos parámetros y pasa los datos a la vista.
`view/graphics.py`	Recibe las series de tiempo y población y dibuja las gráficas comparativas con Matplotlib.
`main.py`	Punto de entrada del programa donde se definen los datos iniciales y se arranca el controlador.

Tabla 4. Parámetros de prueba utilizados en la simulación.

Parámetro	**Descripción**	**Escenario de Crecimiento**	**Escenario de Decrecimiento**
P_0	Población inicial en $t = 0$ (primer valor de `datos`)	1000	1000
r	Tasa de variación poblacional por período	$0{,}13$	$-0{,}10$
σ (`sigma`)	Desviación estándar del ruido aleatorio normal	$45{,}0$	$30{,}0$
Δt (`dt`)	Tamaño del paso para el método continuo de Euler	$0{,}10$	$0{,}10$
`tiempo`	Número total de períodos simulados	6	6

Pasos que ejecuta el programa:

1. Definir en `main.py` el arreglo de entrada `datos` y los parámetros iniciales P_0, r, σ, dt y `tiempo`.
2. Llamar al controlador para instanciar los cuatro modelos a través de `factory/factory.py`.
3. Ejecutar el modelo determinístico evaluando $P(t) = P_0 e^{rt}$ en cada instante t y guardar la lista resultante.
4. Ejecutar el modelo discreto iterando desde 1 hasta `tiempo` con $P_{t+1} = P_t + rP_t$.
5. Ejecutar el modelo estocástico sumando en cada período un valor aleatorio normal$(0, \sigma)$ y aplicando la regla si $P_{t+1} < 0 \Rightarrow P_{t+1} = 0$.
6. Ejecutar el modelo continuo creando el vector de tiempo con paso dt y acumulando $P_{t+dt} = P_t + dt \cdot (rP_t)$.

7. Enviar todos los resultados a `view/graphics.py` para mostrar la gráfica comparativa con título, ejes rotulados, leyenda y cuadrícula.

5. Resultados

5.1. Salida en consola

En la Figura 1 se reserva el espacio para colocar la captura de la terminal al ejecutar `main.py`.

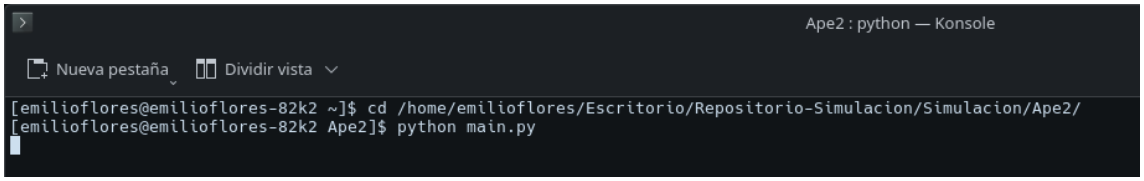


Figura 1: Tabla de resultados mostrada por Matplotlib con los pesos modificados.

5.2. Comparación numérica de los cuatro modelos

En la Tabla 5.2 se comparan los datos reales de la guía frente a los valores calculados por cada modelo en los períodos $t = 0$ a $t = 6$ usando $P_0 = 1000$, $r = 0{,}13$, $dt = 0{,}10$ y una corrida del modelo estocástico con $\sigma = 45{,}0$.

Tabla 5. Comparación de valores poblacionales por período ($P_0 = 1000, r = 0{,}13, dt = 0{,}10$).

Tiempo (t)	Datos Guía	Modelo Determinístico	Modelo Discreto	Modelo Continuo (Euler)	Modelo Estocástico (Ej.)
0	1000	1000.00	1000.00	1000.00	1000.00
1	1100	1138.83	1130.00	1137.87	1108.45
2	1250	1296.93	1276.90	1294.76	1289.12
3	1400	1476.98	1442.90	1473.27	1421.60
4	1600	1682.03	1630.47	1676.40	1655.34
5	1850	1915.54	1842.44	1907.53	1830.18
6	2100	2181.47	2081.95	2170.53	2124.80

5.3. Gráficas comparativas en Matplotlib

En la Figura 2 se muestra la captura de la gráfica generada por `view/graphics.py`.

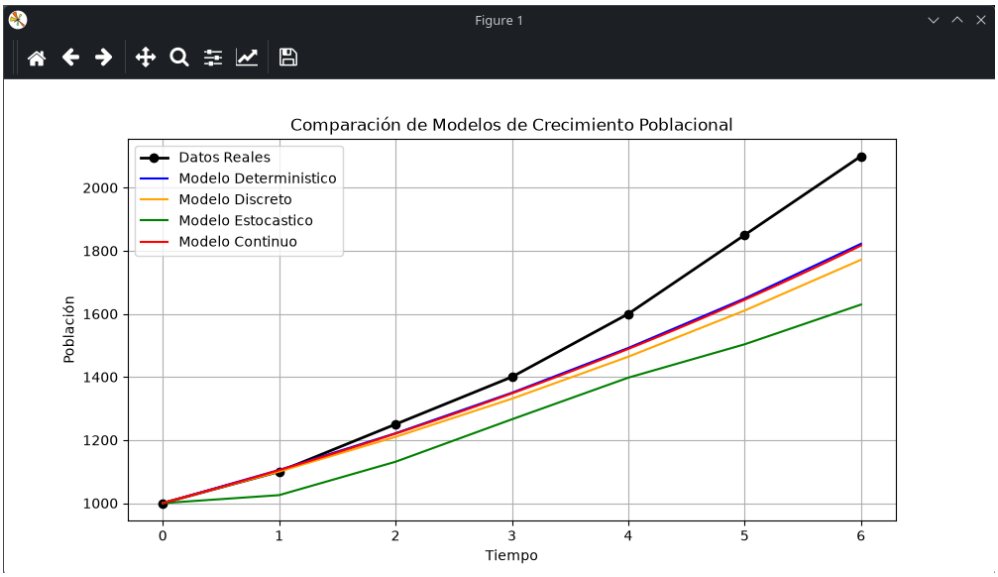


Figura 2: Comparación gráfica de los modelos determinístico, discreto, estocástico y continuo ($r > 0$).

5.4. Análisis de resultados

- **Diferencia entre determinístico, continuo y discreto:** Como se ve en la Tabla 5.2, el modelo determinístico alcanza $2181{,}47$ en $t = 6$, mientras que el discreto llega a $2081{,}95$ (el más cercano al valor final de 2100 de la guía). Esto ocurre porque el modelo discreto solo capitaliza el crecimiento una vez por período, mientras que el modelo continuo de Euler ($2170{,}53$ con $dt = 0{,}10$) actualiza la población diez veces dentro de cada período, acercándose mucho a la curva exponencial exacta.
- **Efecto del ruido en el modelo estocástico:** A diferencia de los otros tres modelos que dan siempre los mismos números al repetir la corrida, el modelo estocástico sube y baja alrededor de la tendencia discreta debido al término $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$. Cuando σ es pequeño la curva casi calca al modelo discreto, pero cuando σ sube se ven picos y caídas pronunciadas en cada período.
- **Influencia de la tasa r:** Al usar una tasa positiva ($r = 0{,}13$) las cuatro curvas suben de forma acelerada; en cambio, al poner una tasa negativa ($r = -0{,}10$) todas las curvas bajan período tras período acercándose a cero.

5.5. Cumplimiento de resultados esperados

Tabla 6. Revisión de los resultados pedidos en la guía de la práctica 002.

Resultado esperado	**Cumplimiento en la práctica**
Modelo implementado correctamente.	Los cuatro modelos fueron programados en Python usando arquitectura MVC y el patrón *Factory Method*.
Gráficas de crecimiento y decrecimiento.	Se generaron en Matplotlib las curvas comparativas tanto para $r > 0$ como para $r < 0$.
Identificación de la influencia de r.	Se comprobó cómo el valor y signo de r definen la velocidad de subida o caída de la población.
Interpretación matemática del comportamiento.	Se contrastó el error de truncamiento de Euler (dt), el salto discreto y el ruido gaussiano (σ).

6. Preguntas de Control

1. **¿Qué representa el arreglo de entrada?**
   Representa las mediciones históricas u observadas del tamaño de la población en cada período de tiempo ($\texttt{datos} = [1000, 1100, 1250, 1400, 1600, 1850, 2100]$), sirviendo como punto de partida ($P_0 = 1000$) y referencia real para comparar los cuatro modelos [1, 2].

2. **¿Cuál es la diferencia entre un modelo determinístico y uno estocástico?**
   El modelo determinístico no tiene azar y siempre produce exactamente el mismo resultado para los mismos datos de entrada; el modelo estocástico incluye variables aleatorias (como el ruido normal ϵ_t), por lo que cada ejecución genera una trayectoria distinta [2, 3].

3. **¿Qué efecto produce modificar la tasa (r)?**
   La tasa r controla la rapidez y dirección del cambio: si $r > 0$ la población crece cada vez más rápido, si $r = 0$ se mantiene constante en P_0, y si $r < 0$ la población disminuye hacia cero [1, 2].

4. **¿Qué representa el parámetro (σ)?**
   Representa la desviación estándar de la distribución normal $\mathcal{N}(0, \sigma^2)$, es decir, la magnitud o intensidad de la variabilidad aleatoria (ruido) que afecta a la población en cada período [1, 3].

5. **¿Qué ocurre cuando aumenta el nivel de ruido?**
   Cuando σ aumenta, las fluctuaciones aleatorias en cada paso se vuelven más grandes y la curva estocástica se aleja con mayor fuerza de la tendencia promedio, pudiendo incluso requerir el corte en cero si ocurre una racha negativa fuerte [1, 3].

6. **¿Cuál es la diferencia entre los modelos discreto y continuo?**
   En el modelo discreto el estado del sistema cambia únicamente en instantes enteros separados ($\Delta t = 1$), mientras que en el modelo continuo el cambio se define en todo instante mediante una ecuación diferencial ($\frac{dP}{dt} = rP$) que se aproxima con pasos pequeños dt [2, 3].

7. **¿Qué función cumple (dt)?**
   En el método numérico de Euler, dt (Δt) es el tamaño del paso de integración; mientras más pequeño sea dt, más cerca estará la aproximación numérica de la curva exponencial exacta, aunque requiere más iteraciones de cálculo [1, 4].

8. **¿Por qué es importante visualizar los resultados mediante gráficas?**
   Porque al superponer las curvas en una misma gráfica es mucho más fácil detectar de un vistazo cómo se separan los modelos con el paso del tiempo, cuánto desvía el ruido estocástico a la población y qué tan bien se acerca Euler al modelo exacto [2, 5].

9. **¿Qué modelo presenta comportamiento aleatorio?**
   El **modelo estocástico**, ya que es el único de los cuatro que suma un término aleatorio $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$ en cada período de la simulación [1].

10. **¿Cómo cambia la simulación cuando se introduce un nuevo arreglo de datos?**
    Al cambiar el arreglo de entrada cambian la población inicial P_0, el número de perío-

dos a simular y la tasa de crecimiento r estimada, desplazando la escala vertical y la inclinación de todas las curvas generadas por el programa [1, 2].

7. Conclusiones

- Se implementaron y compararon los cuatro paradigmas de simulación en Python organizando el código con arquitectura MVC y el patrón *Factory Method*, lo que permitió mantener cada modelo matemático aislado en su propio módulo dentro del paquete `factory/`.
- Se comprobó que el modelo continuo de Euler con un paso pequeño ($dt = 0{,}10$) reproduce casi con exactitud la curva del modelo determinístico exponencial, mientras que el modelo discreto crece un poco por debajo al actualizar el incremento solo una vez por período.
- A través del modelo estocástico se verificó cómo el ruido gaussiano (σ) introduce variaciones reales en cada corrida y por qué es necesario validar que la población no tome valores negativos cuando aumenta la volatilidad.

8. Bibliografía / Referencias

1. J. O. Guamán Q., *Guía de Actividades Práctico-Experimentales Nro. 002: Comparación de modelos determinísticos, estocásticos, discretos y continuos*. Carrera de Computación, FEIRNNR, Universidad Nacional de Loja, 2026.
2. Asignatura Simulación, *Diapositivas de la Semana 2: Paradigmas de Simulación: Modelos Determinísticos, Estocásticos, Discretos y Continuos*. Carrera de Computación, FEIRNNR - UNL, 2026.
3. A. M. Law, *Simulation Modeling and Analysis*, 5.ª ed. New York, NY, EE. UU.: McGraw-Hill Education, 2014.
4. S. C. Chapra y R. P. Canale, *Métodos numéricos para ingenieros*, 7.ª ed. México D. F.: McGraw-Hill, 2015.
5. J. D. Hunter, "Matplotlib: A 2D Graphics Environment", *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. Disponible en: `https://matplotlib.org`

9. Anexos

Anexo A. Instalación y ejecución

Repositorio del proyecto: Repositorio GitHub

```
git clone https://github.com/NullDistortion/Repositorio-Simulacion.git
cd Repositorio-Simulacion/ape2
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
python main.py
```

Anexo B. Declaración de uso de inteligencia artificial

Siguiendo los principios de transparencia y honestidad de la asignatura, declaro que durante el desarrollo de esta práctica utilicé herramientas de inteligencia artificial generativa como apoyo para estructurar la arquitectura del código en Python, aplicar el patrón de diseño creacional *Factory Method* junto con el modelo MVC, generar la vista comparativa en Matplotlib.

Prompt utilizado para el desarrollo de la arquitectura y diseño del código:

> *"Partiendo del pseudocódigo para los cuatro modelos de crecimiento poblacional (determinístico, discreto, estocástico y continuo mediante Euler), implementa el proyecto en Python aplicando una arquitectura Modelo-Vista-Controlador (MVC) combinada con el patrón de diseño creacional Factory Method. Separa cada modelo matemático en su propio archivo dentro de un paquete factory (`deterministic.py`, `discreet.py`, `stochastic.py` y `continuous.py`) junto con su clase fábrica (`factory.py`), de modo que el controlador (`controller.py`) instancie y ejecute los cuatro modelos sin acoplarse a sus fórmulas internas y envíe los resultados a la vista gráfica (`graphics.py`) en Matplotlib, respetando en todo momento las normas de Clean Code y los principios SOLID."*

Declaración de Delegación en Inteligencia Artificial Generativa (Traducción GAIDeT):

Los autores declaran el uso de inteligencia artificial generativa en el proceso de investigación y redacción. De acuerdo con la taxonomía GAIDeT (2026), las siguientes tareas fueron delegadas a herramientas de IAG bajo supervisión humana total:

- Generación de ideas (*Idea generation*)
- Generación de código (*Code generation*)
- Optimización de código (*Code optimization*)
- Automatización de procesos (*Process automation*)
- Creación de algoritmos para análisis de datos (*Creation of algorithms for data analysis*)
- Validación (*Validation*)
- Limpieza de datos (*Data cleaning*)
- Análisis de datos (*Data analysis*)
- Visualización (*Visualization*)

La herramienta de IAG utilizada fue: **Gemini Argon Alto y NotebookLM**.
La responsabilidad del manuscrito final recae íntegramente en los autores.
Las herramientas de IAG no figuran como autores y no asumen responsabilidad por los resultados finales.
Declaración presentada por: Emilio Josue Flores Gallardo.